Supplementary Appendix

TreeFlow: Probabilistic Modelling and Automatic Differentiation for Phylogenetics

Swanepoel et al.

Model specification

TreeFlow provides two ways to specify phylogenetic models. For standard phylogenetic analyses, models can be defined using a YAML-based model definition format that is used as input to TreeFlow's command line interfaces. For more flexible model specification, the Python developer API allows users to directly compose TensorFlow Probability distributions into a joint distribution object.

YAML model definition format

The YAML model definition format provides a concise, text-based specification of standard phylogenetic models. The format is a nested dictionary with keys at various levels corresponding to model components (tree prior, substitution model, clock model, site model), their parameters, and the prior distributions on those parameters. Figure 1 shows an example YAML model definition for a phylogenetic analysis of influenza sequences, specifying a coalescent tree prior, strict molecular clock, discretized Gamma site rate model, and GTR substitution model with independent Gamma priors on the relative substitution rates.

Python API model specification

For models that go beyond the standard form supported by the YAML format, the Python developer API provides direct access to TensorFlow Probability's joint distribution framework. This allows users to compose any combination of distributions and transformations into a probabilistic graphical model. Figure 2 shows an example of this flexibility: the code specifies a phylogenetic model for the carnivores dataset, with highlighted lines showing the small changes required to extend the model from a single transition-transversion ratio (kappa) to a per-lineage kappa parameter. This extension, which leverages TensorFlow's vectorization and broadcasting functionality, would be difficult to express in a fixed model definition format.

For further examples and a detailed tutorial on using TreeFlow's Python API and command line interfaces, see the TreeFlow documentation at `https://treeflow.readthedocs.io`.

```yaml
clock:
  strict:
    clock_rate:
      lognormal:
        loc: -2.0
        scale: 2.0
site:
  discrete_gamma:
    category_count: 4
    site_gamma_shape:
      lognormal:
        loc: 0.0
        scale: 1.0
substitution:
  gtr_rel:
    frequencies:
      dirichlet:
        concentration:
        - 2.0
        - 2.0
        - 2.0
        - 2.0
    rate_ac:
      gamma:
        concentration: 0.05
        rate: 0.05
    rate_ag:
      gamma:
        concentration: 0.05
        rate: 0.05
    rate_at:
      gamma:
        concentration: 0.05
        rate: 0.05
    rate_cg:
      gamma:
        concentration: 0.05
        rate: 0.05
    rate_gt:
      gamma:
        concentration: 0.05
        rate: 0.05
tree:
  coalescent:
    pop_size:
      lognormal:
        loc: 1.0
        scale: 1.5
```

Figure 1: YAML model definition for the analysis of the influenza dataset. The top-level keys specify the clock model, site rate model, substitution model, and tree prior. Each model component's parameters are given prior distributions from the standard distributions available in TensorFlow Probability.

```
site_category_count = 4
pattern_counts = alignment.get_weights_tensor()
subst_model = HKY()

def build_sequence_dist(tree, kappa, frequencies, site_gamma_shape):
    unrooted_tree = tree.get_unrooted_tree()
    site_rate_distribution = DiscretizedDistribution(
        category_count=site_category_count,
        distribution=Gamma(
            concentration=site_gamma_shape,
            rate=site_gamma_shape
        ),
    )
    transition_probs_tree = get_transition_probabilities_tree(
        unrooted_tree,
        subst_model,
        rate_categories=site_rate_distribution.normalised_support,
        frequencies=frequencies,
        frequencies=tf.broadcast_to(
            tf.expand_dims(frequencies, -2),
            kappa.shape + (4,)
        ),
        kappa=kappa
    )
    return SampleWeighted(
        DiscreteParameterMixture(
            site_rate_distribution,
            LeafCTMC(
                transition_probs_tree,
                expand_dims(frequencies, -2),
            ),
        ),
        sample_shape=alignment.site_count,
        weights=pattern_counts
    )

model = JointDistributionNamed(dict(
    birth_rate=LogNormal(c(1.0), c(1.5)),
    tree=lambda birth_rate: Yule(tree.taxon_count, birth_rate),
    kappa=LogNormal(c(0.0), c(2.0)),
    kappa=Sample(
        LogNormal(c(0.0), c(2.0)),
        tree.branch_lengths.shape
    ),
    site_gamma_shape=LogNormal(c(0.0), c(1.0)),
    frequencies=Dirichlet(c([2.0, 2.0, 2.0, 2.0])),
    sequences=build_sequence_dist
))
```

Figure 2: Code to specify models for the carnivores analysis using TreeFlow's Python API. Highlighted lines show changes (from previous line) to add kappa variation across lineages. The `Sample` distribution is used to make the kappa prior an independent identically-distributed sample for each branch. The frequencies parameter needs to be manually broadcasted over the added kappa dimension.